

# Task 1: LoRA Rank Ablation Study with MLX-LM

TinyLlama Instruction Tuning Project

March 22, 2026

## Task description

We study how LoRA rank affects cost and quality on Apple Silicon: train **five** TinyLlama models with ranks 4, 8, 16, 32, 64 and fixed  $\alpha/r = 2$ . For each run we record **wall-clock training time**, **peak Metal memory**, **final test loss**, and **inference latency** (ms/token, tok/s) over **100** Alpaca-format test prompts. We fill a comparison matrix with rank, throughput, memory, and **MMLU-style** accuracy (4-option MCQ), identify the **Pareto-optimal** rank using **accuracy vs. latency** only, and save matplotlib .png plots for reporting.

## Procedure & technical implementation

1. **Environment.** Python 3, MLX / mlx-lm; Apple Silicon with Metal. Entry point: `task1_lora_rank_ablation.py` (also make `task1`).
2. **LoRA configs.** Base model TinyLlama/TinyLlama-1.1B-Chat-v1.0; training data path `./data/`; LoRA modules `self_attn.q_proj`, `self_attn.v_proj`; rank  $r$ ;  $\alpha = 2r$ , `scale =  $\alpha/r = 2$` ; batch 2; default 2000 iterations; LR  $2 \times 10^{-4}$ ; YAML per rank under `results/task1/`.
3. **Metrics from training.** Subprocess isolation for fair peak memory; regex parse of test loss from `mlx_lm` logs; `train_time_sec` wall clock.
4. **Inference benchmark.** Load base + adapter; read first 100 user messages from `data/test.jsonl`; warm-up; generate up to 100 tokens each; compute global ms/token and tok/s.
5. **MMLU-style accuracy.** `evaluate_mmlu_accuracy.py`: for each of 100 test rows, build one MCQ with the gold answer plus three distractors from other rows; model must reply with A/B/C/D; `accuracy = fraction correct  $\times 100$` . Stored in the comparison matrix and `accuracy_results.json`.
6. **Pareto (2D).** Frontier: maximize MCQ accuracy, minimize `ms_per_token`; memory *excluded*. Among non-dominated points, select rank with 0.5 normalized accuracy + 0.5 normalized latency. Artifacts: `comparison_matrix.json`, `pareto.json`, `rank_ablation_metrics.png`, `memory_speed_curves.png`, `pareto_analysis.png`.

## Results

**Quantitative summary.** Table 1 uses values from `results/task1/comparison_matrix.json` (training times converted to minutes). MCQ columns in JSON may be zero until `evaluate_mmlu_accuracy.py` is run after the MCQ update—re-run the task script to populate. Current 2D Pareto choice: rank **8** (`pareto.json`). Ranks 8–16 achieve the best test loss in this run; rank 8 leads in tok/s.

Table 1: LoRA rank ablation (checked-in `comparison_matrix.json`).

| $r$ | Time (min) | Mem (MB) | Test loss | tok/s | ms/tok | MCQ % |
|-----|------------|----------|-----------|-------|--------|-------|
| 4   | 19.3       | 3057     | 1.263     | 96.8  | 10.33  | —     |
| 8   | 18.9       | 3066     | 1.262     | 99.1  | 10.09  | —     |
| 16  | 18.7       | 3075     | 1.262     | 98.9  | 10.12  | —     |
| 32  | 19.2       | 3100     | 1.265     | 97.5  | 10.26  | —     |
| 64  | 19.5       | 3151     | 1.270     | 95.7  | 10.45  | —     |

“—” = run `evaluate_mmlu_accuracy.py` after training to fill MCQ %; script also writes `accuracy_results.csv`.

## Evidence (screenshots)

**Figure 1** is a matplotlib export saved by the pipeline: three panels (peak memory, inference speed, test loss vs. rank). Running the script also writes `memory_speed_curves.png` and `pareto_analysis.png` (accuracy–latency scatter with frontier); add those files to `results/task1/` for the full figure set.

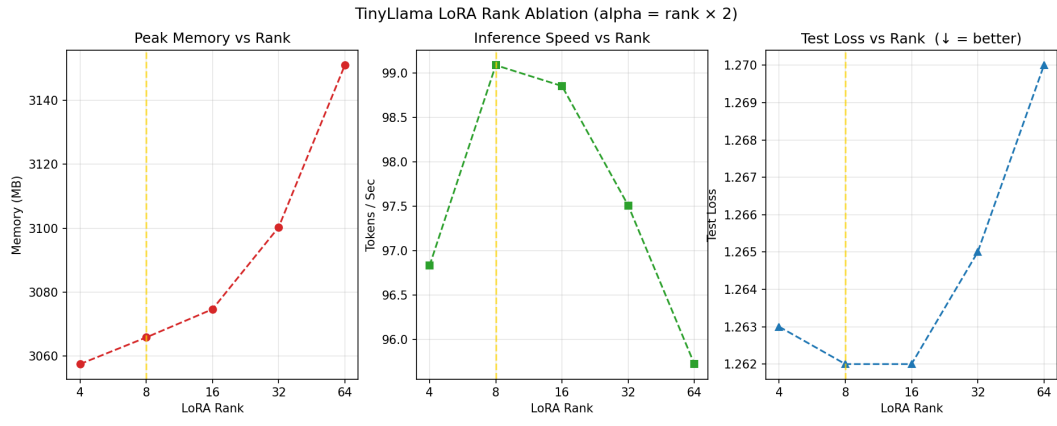


Figure 1: Task 1 visualization: memory (left), throughput (center), test loss (right) vs. LoRA rank. Regenerate with `make task1`.